Lab8

Course: ARTI 404 – Image Processing

Student Name: Muhannad Almutlaq

Student ID: 2240006060

Procedural Task#1

Detect the edges of the kid1.jpg photo with the Sobel operator. Load the image, convert it to grayscale, and smooth it with a 5x5 Gaussian blur to reduce noise (Sobel can also be tried on the original image or the plain grayscale image). Compute the horizontal and vertical derivatives with cv2.Sobel using a 3x3 kernel and cv2.CV_64F for high precision, combine them with cv2.magnitude to get the final edge map, and convert it to 8-bit. Show the original photo and the Sobel edge map side by side.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

imgFile = 'kid1.jpg'

# 1. Load the real image

img = cv2.imread(imgFile)

# 2. Convert to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Apply Gaussian Blur to reduce noise
# (5, 5) is the kernel size; 0 means standard deviation is calculated from kernel size

# ----------| You can try Sobel with Original Image, Gray scale and Blur
# ----------| Try these
# ---(1)---|  blurred = img
# ---(2)---|  blurred = gray

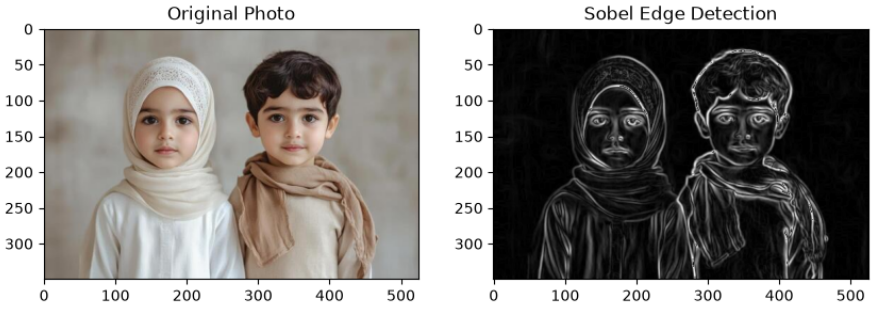
blurred = cv2.GaussianBlur(gray, (5, 5), 0) # blurred = img or gray   ## Check the
results

# 4. Apply Sobel filter
# cv2.CV_64F handles the high precision needed for derivatives
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
```

```python
# 5. Calculate Magnitude (Final Edge Map)
sobel_combined = cv2.magnitude(sobelx, sobely)

# 6. Normalize to 0-255 for display
sobel_final = np.uint8(np.absolute(sobel_combined))

# Visualization
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),
plt.title('Original Photo')
plt.subplot(1, 2, 2), plt.imshow(sobel_final, cmap='gray'), plt.title('Sobel Edge
Detection')
plt.show()
```



Procedural Task#2

Detect the edges with the Roberts cross operator. Load the image in grayscale and blur it with a 5x5 Gaussian filter first, because the 2x2 Roberts kernels are very sensitive to noise. Define the two Roberts kernels [[1, 0], [0, -1]] and [[0, 1], [-1, 0]], apply them with cv2.filter2D using cv2.CV_64F, combine the two responses with cv2.magnitude, and convert the result to 8-bit. Compared with Sobel, Roberts is faster and produces thinner, sharper edges, but it has poor noise resistance. Show the grayscale image and the Roberts edge map side by side.

```
# 1. Load a real image in grayscale
image = cv2.imread(imgFile, 0) #cv2.IMREAD_GRAYSCALE)

# 2. Pre-process: Always blur for Roberts!
# Its 2x2 kernel is very sensitive to noise.
blurred = cv2.GaussianBlur(image, (5, 5), 0)

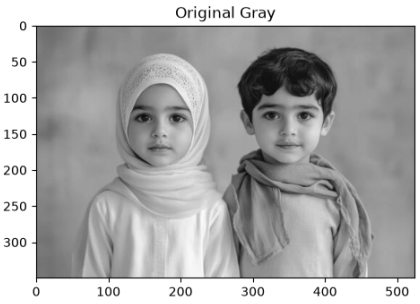
# 3. Define Roberts Cross kernels
kernel_x = np.array([[1, 0], [0, -1]], dtype=np.float64)
kernel_y = np.array([[0, 1], [-1, 0]], dtype=np.float64)

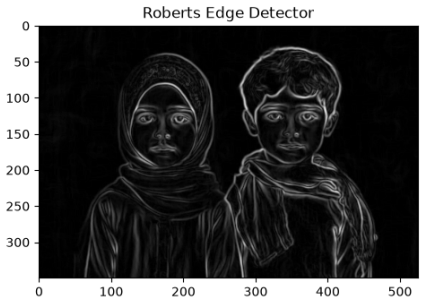
# 4. Apply kernels using convolution
roberts_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
roberts_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)

# 5. Calculate Magnitude (Combine X and Y)
# Magnitude = sqrt(Gx^2 + Gy^2)
roberts_combined = cv2.magnitude(roberts_x, roberts_y)

# 6. Convert to uint8 for display
roberts_final = np.uint8(np.absolute(roberts_combined))

# Display
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(image, cmap='gray'), plt.title('Original Gray')
plt.subplot(1, 2, 2), plt.imshow(roberts_final, cmap='gray'), plt.title('Roberts Edge
Detector')
plt.show()
```


Original Gray
0
50
100
150
200
250
300
0
100
200
300
400
500


Roberts Edge Detector
0
50
100
150
200
250
300
0
100
200
300
400
500

Procedural Task#3

Detect the edges with the Prewitt operator. Load the color image, convert it to grayscale, and apply a 5x5 Gaussian blur to reduce grain. Define the 3x3 Prewitt kernels for the horizontal and vertical directions manually, apply them with cv2.filter2D using cv2.CV_64F, combine the two responses with cv2.magnitude, and convert the result to 8-bit. Show the original photo and the Prewitt edge map side by side.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load a real image in grayscale
img = cv2.imread(imgFile)

# 2. Grayscale & Blur (Essential for real photos to reduce grain)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

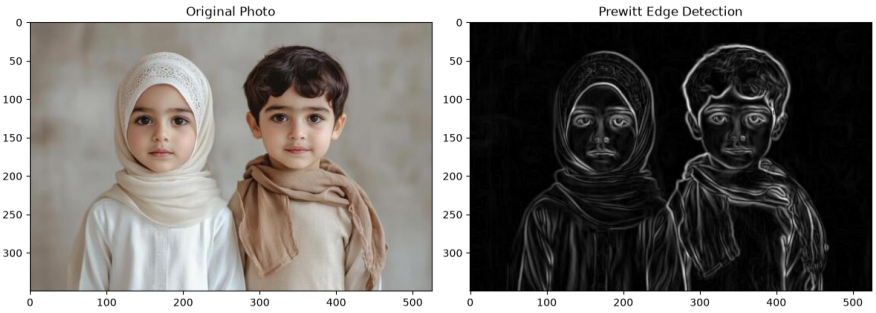
# 3. Define Prewitt Kernels manually
kernel_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64)
kernel_y = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64)

# 4. Apply filters
prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)

# 5. Calculate Magnitude (Combine X and Y)
prewitt_combined = cv2.magnitude(prewitt_x, prewitt_y)

# 6. Normalize to 8-bit for display
prewitt_final = np.uint8(np.absolute(prewitt_combined))

# Visualizing results
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),
plt.title('Original Photo')
plt.subplot(1, 2, 2), plt.imshow(prewitt_final, cmap='gray'), plt.title('Prewitt Edge
Detection')
plt.tight_layout()
plt.show()
```


Original Photo
Prewitt Edge Detection
0
50
100
150
200
250
300
0
100
200
300
400
500

Procedural Task#4

Detect the edges with the Canny edge detector. Load the color image, convert it to grayscale, and apply cv2.Canny with a low threshold of 100 and a high threshold of 200 (a 1:2 or 1:3 ratio between the thresholds usually works best). Show the original image and the Canny edge map side by side.

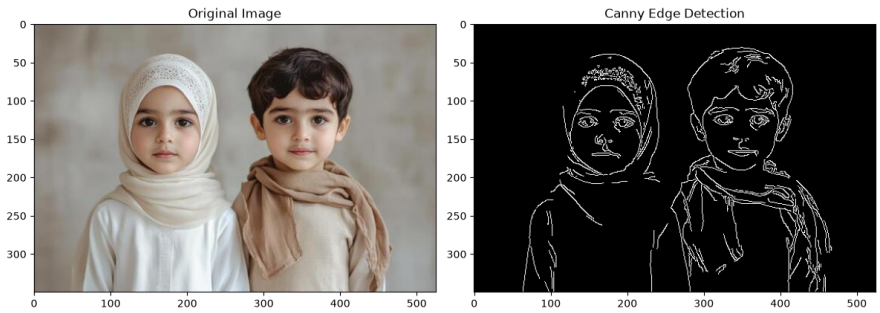
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load the real image
# Replace 'building.jpg' with your file path
img = cv2.imread(imgFile)


# 2. Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Apply Canny Edge Detection
# Syntax: cv2.Canny(image, low_threshold, high_threshold)
# Tip: A 1:2 or 1:3 ratio for thresholds usually works best
canny_edges = cv2.Canny(gray, 100, 200)

# 4. Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),
plt.title('Original Image')
plt.subplot(1, 2, 2), plt.imshow(canny_edges, cmap='gray'), plt.title('Canny Edge
Detection')
plt.tight_layout()
plt.show()
```



Procedural Task#5

Improve Canny edge detection for color images by applying it to every color channel. Define a function that loads the color image, splits it into its B, G, and R channels, blurs each channel with a 5x5 Gaussian filter, and applies cv2.Canny with thresholds 50 and 150 to each one. Merge the three edge maps with cv2.bitwise_or so that an edge found in any of the three channels is kept. Run the function on kid1.jpg and show the original color image and the combined edge map side by side.

```
def best_color_edge_detector(image_path):
    # 1. Load the color image
    img = cv2.imread(image_path)
    if img is None:
        return "Image not found."

    # 2. Split into B, G, R channels
    channels = cv2.split(img)
    edge_maps = []

    for channel in channels:
        # 3. Pre-process each channel with Gaussian Blur to reduce noise
        blurred = cv2.GaussianBlur(channel, (5, 5), 0)

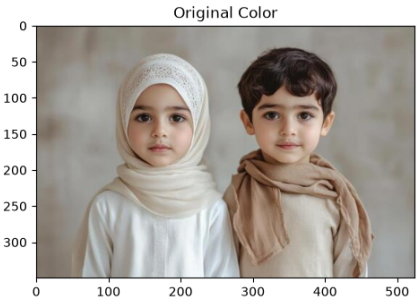
        # 4. Apply Canny to each channel
        # 50 and 150 are the low and high thresholds
        edges = cv2.Canny(blurred, 50, 150)
        edge_maps.append(edges)

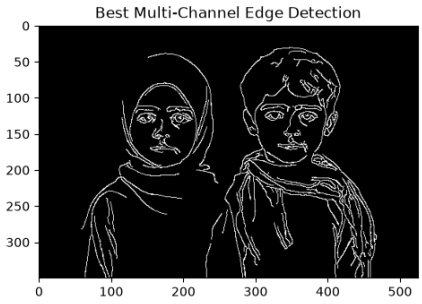
    # 5. Merge edges from all channels using bitwise OR
    # This keeps any edge found in ANY of the three color layers
    combined_edges = cv2.bitwise_or(edge_maps[0], edge_maps[1])
    combined_edges = cv2.bitwise_or(combined_edges, edge_maps[2])

    return img, combined_edges

# Run the detector
original, edges = best_color_edge_detector(imgFile)

# Display results
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB)),
plt.title('Original Color')
plt.subplot(1, 2, 2), plt.imshow(edges, cmap='gray'), plt.title('Best Multi-Channel
Edge Detection')
plt.show()
```


Original Color
0
50
100
150
200
250
300
0
100
200
300
400
500


Best Multi-Channel Edge Detection
0
50
100
150
200
250
300
0
100
200
300
400
500

Procedural Task#6

Apply the Laplacian of Gaussian (LoG) filter. Load the image in grayscale, smooth it with a 5x5 Gaussian blur, and then apply the Laplacian operator with cv2.Laplacian using ksize=5 and cv2.CV_64F so that negative values (needed for zero-crossings) are preserved. Take the absolute value and convert the result to 8-bit for display. Show the grayscale image and the LoG-filtered image side by side.

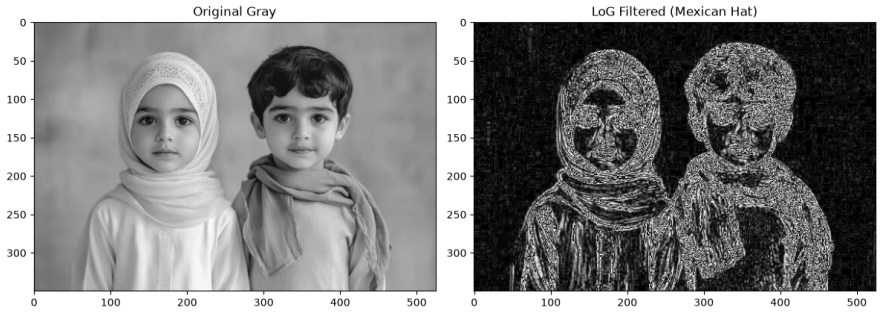
```
# 1. Load a real-world photo in grayscale
img = cv2.imread(imgFile, cv2.IMREAD_GRAYSCALE)

# 2. Step 1: Gaussian Blur (Smoothing)
# Larger kernel/sigma reduces more noise but may miss small edges
blurred = cv2.GaussianBlur(img, (5, 5), 0)

# 3. Step 2: Laplacian Operator (Second Derivative)
# cv2.CV_64F is used to store negative values (crucial for zero-crossings)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F, ksize=5)

# 4. Post-processing: Normalize for display
# Since the Laplacian has negative and positive values, we take the absolute value
log_final = np.uint8(np.absolute(laplacian))

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original Gray')
plt.subplot(1, 2, 2), plt.imshow(log_final, cmap='gray'), plt.title('LoG Filtered
(Mexican Hat)')
plt.tight_layout()
plt.show()
```



Procedural Task#7

Approximate the LoG filter with the Difference of Gaussians (DoG). Load the image in grayscale and blur it twice with cv2.GaussianBlur using a narrow sigma (1.0) and a wide sigma (2.0). Subtract the wide blur from the narrow blur using float32 to avoid overflow and underflow, then map the result to the 0-255 range with cv2.normalize. A simple threshold at 127 also converts it into a binary edge map. Show the original image and the normalized DoG result side by side.

```
# 1. Load the real-world photo
img = cv2.imread(imgFile, cv2.IMREAD_GRAYSCALE)

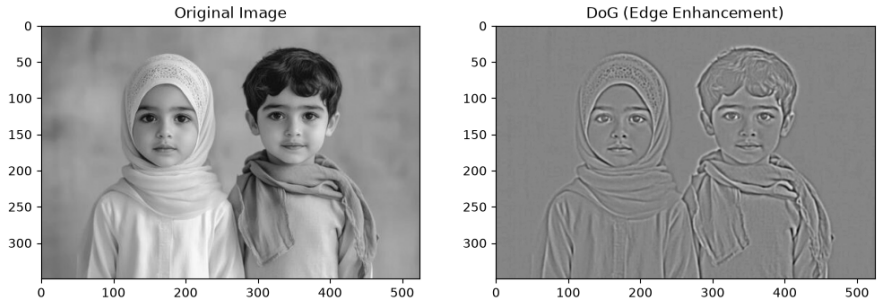
# 2. Apply two Gaussian blurs with different standard deviations (Sigmas)
# Low sigma preserves detail; High sigma captures broad structure
blur_narrow = cv2.GaussianBlur(img, (0, 0), sigmaX=1.0)
blur_wide = cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)

# 3. Calculate the Difference (DoG)
# Note: We use float32 to avoid overflow/underflow during subtraction
dog = blur_narrow.astype(np.float32) - blur_wide.astype(np.float32)

# 4. Normalize or Threshold for visibility
# Option A: Map to 0-255 range
dog_normalized = cv2.normalize(dog, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Option B: Simple threshold to create a binary edge map
_, dog_binary = cv2.threshold(dog_normalized, 127, 255, cv2.THRESH_BINARY)

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(1, 2, 2), plt.imshow(dog_normalized, cmap='gray'), plt.title('DoG (Edge
Enhancement)')
plt.show()
```



Procedural Task#8

Detect edges in a color image with a gradient computed on every color channel. Split the image into its B, G, and R channels, blur each one with a 3x3 Gaussian filter, and compute its Sobel gradient magnitude with a helper function. Combine the three gradients as the square root of the sum of their squares so that edges caused only by color transitions are also captured, normalize the result to 0-255, and display it with the inferno colormap. Show the original color image and the color-aware gradient map side by side.

```
# 1. Load the real-world color image
image = cv2.imread(imgFile)

# 2. Split the image into its R, G, and B components
# Note: OpenCV uses BGR order by default
b, g, r = cv2.split(image)

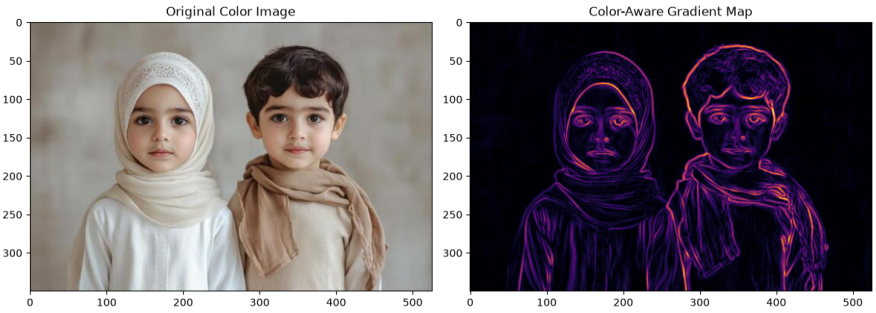
def get_channel_gradient(channel):
    # Use Sobel for gradient calculation on a single channel
    # Blur first to reduce noise in that specific color layer
    blurred = cv2.GaussianBlur(channel, (3, 3), 0)
    gx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
    return cv2.magnitude(gx, gy)

# 3. Compute gradients for each color channel
grad_b = get_channel_gradient(b)
grad_g = get_channel_gradient(g)
grad_r = get_channel_gradient(r)

# 4. Combine gradients from all channels
# This ensures edges from color-only transitions are captured
combined_gradient = np.sqrt(grad_b**2 + grad_g**2 + grad_r**2)

# 5. Normalize to 0-255 for visualization
color_edges = cv2.normalize(combined_gradient, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)),
plt.title('Original Color Image')
plt.subplot(1, 2, 2), plt.imshow(color_edges, cmap='inferno'), plt.title('Color-Aware
Gradient Map')
plt.tight_layout()
plt.show()
```


Original Color Image
Color-Aware Gradient Map
0
50
100
150
200
250
300
0
100
200
300
400
500

Task#1

Apply and compare edge detection methods on a new image. Select Mercedes_Automobile.png instead of the image used in the original notebook, load it, and convert it to RGB (for display) and to grayscale. Smooth the grayscale image with a 5x5 Gaussian blur to reduce noise, then apply the Sobel detector (cv2.Sobel with a 3x3 kernel and cv2.CV_64F, combined with cv2.magnitude), the Prewitt detector (custom 3x3 kernels applied with cv2.filter2D, combined with cv2.magnitude), and the Canny detector (cv2.Canny with thresholds 50 and 150). Display the original image and the Sobel, Prewitt, and Canny edge maps in a 2x2 grid.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_bgr= cv2.imread("./Mercedes Automobile.png")
img_rgb= cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
gray= cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Apply Gaussian Blur to reduce noise before edge detection
blurred= cv2.GaussianBlur(gray, (5, 5), 0)

# Sobel Edge Detector
sobelx= cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize= 3)
sobely= cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize= 3)
sobel_edges= np.uint8(np.absolute(cv2.magnitude(sobelx, sobely)))

# Prewitt Edge Detector. It uses custom kernels for X and Y derivatives
kernelx= np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
kernely= np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]])
prewittx= cv2.filter2D(blurred, cv2.CV_64F, kernelx)
prewitty= cv2.filter2D(blurred, cv2.CV_64F, kernely)
prewitt_edges= np.uint8(np.absolute(cv2.magnitude(prewittx, prewitty)))

# Canny Edge Detector
canny_edges= cv2.Canny(blurred, 50, 150)

# Display the original image and the three resulting edge maps
plt.figure(figsize= (15, 10))

plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")

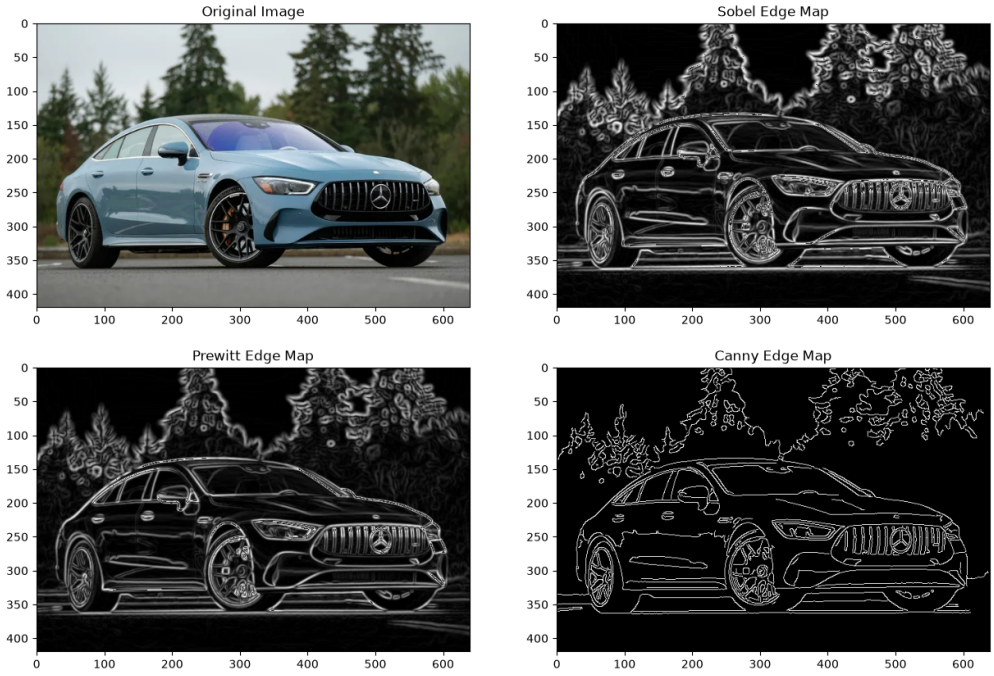
plt.subplot(2, 2, 2)
plt.imshow(sobel_edges, cmap= "gray")
```

```
plt.title("Sobel Edge Map")

plt.subplot(2, 2, 3)
plt.imshow(prewitt_edges, cmap= "gray")
plt.title("Prewitt Edge Map")

plt.subplot(2, 2, 4)
plt.imshow(canny_edges, cmap= "gray")
plt.title("Canny Edge Map")

plt.show()
```



Task#2

Investigate the effect of the Canny thresholds. Apply cv2.Canny to the blurred grayscale image using three threshold pairs: low = 30 with high = 90, low = 50 with high = 150, and low = 100 with high = 200. Display the three resulting edge maps side by side and explain how changing the thresholds affects the number of detected edges and the visibility of image details.

```
# Try to apply Canny edge detection using the specified threshold pairs
canny_30_90= cv2.Canny(blurred, 30, 90)
canny_50_150= cv2.Canny(blurred, 50, 150)
canny_100_200= cv2.Canny(blurred, 100, 200)

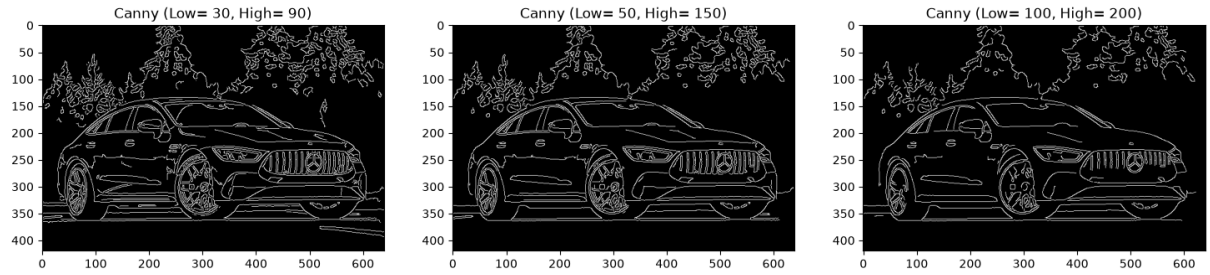
# Display the three resulting edge maps side by side
plt.figure(figsize= (18, 5))

plt.subplot(1, 3, 1)
plt.imshow(canny_30_90, cmap= "gray")
plt.title("Canny (Low= 30, High= 90)")

plt.subplot(1, 3, 2)
plt.imshow(canny_50_150, cmap= "gray")
plt.title("Canny (Low= 50, High= 150)")

plt.subplot(1, 3, 3)
plt.imshow(canny_100_200, cmap= "gray")
plt.title("Canny (Low= 100, High= 200)")

plt.show()
```



Explanation

In Canny edge detection, the high threshold decides which strong edges are kept for sure, and the low threshold decides which weaker edges are kept only when they are connected to a strong edge. Raising both thresholds therefore makes the detector more selective, and lowering them makes it more sensitive.

With the lowest pair (30, 90), the edge map contains the most edges: the car outline, the grille, the wheels, and the body lines are all clearly visible, but many weak and noisy edges also appear, especially in the textured trees in the background and on the road, which make the result cluttered. With the middle pair (50, 150), most of the important car edges are still present while a large part of the background clutter disappears, so it gives a good balance between detail and noise. With the highest pair (100, 200), the edge map is the cleanest and has the fewest edges, but some weak edges such as the fine body lines, the reflections, and parts of the background are lost, so some image details become less visible or disappear.

Counting the edge pixels confirms this trend: about 23,500 pixels (8.8%) for (30, 90), about 19,500 pixels (7.3%) for (50, 150), and about 15,400 pixels (5.7%) for (100, 200). In summary, lower thresholds detect more edges and more detail but also more noise, while higher thresholds detect fewer edges, suppress noise, and keep only the strongest edges.